

```
import cv2
import numpy as np
from google.colab import drive
from google.colab.patches import cv2_imshow
import matplotlib.pyplot as plt
```

```
from google.colab import files
uploaded = files.upload()
```

```
img = cv2.imread('/content/zzz.jpg',cv2.IMREAD_GRAYSCALE)
cv2_imshow(img)
img2 = '/content/yyy.jpeg'
rgb_image = cv2.imread(img2)
cv2_imshow(rgb_image)
```

1)

```
#1 - Grayscale to Binary (Mean threshold)
```

```
def grayscale_to_binary_mean_threshold(greyscale_img):
    mean_intensity = np.mean(greyscale_img)
    print(f"Mean Intensity (Threshold): {mean_intensity:.2f}")
    _, binary_img = cv2.threshold(greyscale_img, mean_intensity, 255,
cv2.THRESH_BINARY)
    return binary_img
```

```
binary_image_mean = grayscale_to_binary_mean_threshold(img)
print("Binary Image (Mean Threshold):")
cv2_imshow(binary_image_mean)
```

```
#2 Grayscale to Binary (User threshold)
```

```
user_threshold = int(input("Enter the threshold value:"))
binary_user = cv2.threshold(img,user_threshold,255,cv2.THRESH_BINARY)[1]
print(f"User threshold used:{user_threshold}")
cv2_imshow(binary_user)
```

2)

```
# RGB to grayscale (Using mean)
def rgb_to_grayscale_mean(rgb_image):
    grayscale_image = np.mean(rgb_image, axis=2).astype(np.uint8)
    return grayscale_image

grayscale_image = rgb_to_grayscale_mean(rgb_image)
cv2_imshow(grayscale_image)

# RGB to greyscale using weigtage of 3 planes
img_rgb = cv2.imread(img2)
B,R,G = cv2.split(img_rgb)
gray_mean=( (B.astype(np.float64)+G.astype(np.float64)+R.astype(np.float64)
)/3).astype(np.uint8)
wr,wg,wb=0.3,0.4,0.3
gray_weighted=(wb*B+wg*G+wr*R).astype(np.uint8)
print("Grayscale Image")
cv2_imshow(gray_weighted)
```

3)

```
#Image complement
print("Original Image")
cv2_imshow(img)
complement_gray = 255 - img
print("Complement Image")
cv2_imshow(complement_gray)

#complement of binary
# convert grayscale to binary first

threshold = int (input ())
-, binary = cv2.threshold(img,threshold,255,cv2.THRESH_BINARY)
cv2.imshow(binary)

binarycomplement = cv2.bitwise_not(binary)
```

4)

```
#Log transform
print("Original Image")
cv2_imshow(img)
print("Log Transform Enhanced Image:")
```

```

img_float = img.astype(np.float32)
log_transformed = 1 * np.log1p(img_float)
log_normalised = cv2.normalize(log_transformed, None, 0, 255,
cv2.NORM_MINMAX)
log_enhanced = log_normalised.astype(np.uint8)
cv2_imshow(log_enhanced)

```

5) *#Gamma transform*

```

gamma = 2
gamma_img = np.array(255*(img / 255) ** gamma, dtype = 'uint8')
print("Gamma Image")
cv2_imshow(gamma_img)

```

6)

```

#contrast streching
print("Original Image")
cv2_imshow(img)
print("Contrast streching")
img_float = img.astype(np.float32)
min_val = np.min(img_float)
max_val = np.max(img_float)
contrast_stretched = (img_float - min_val) * (255 / (max_val - min_val))
contrast_stretched = contrast_stretched.astype(np.uint8)
cv2_imshow(contrast_stretched)

```

7) *#histogram equalization*

```

print("Original Image")
cv2_imshow(img)
print("Histogram Equalization")
equalized_img = cv2.equalizeHist(img)
cv2_imshow(equalized_img)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.hist(img.ravel(), 256, [0, 256])
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity')

```

```
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.hist(equalized_img.ravel(), 256, [0, 256])
plt.title('Equalised Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```

8)

```
#histogram matching
import numpy as np
import matplotlib.pyplot as plt
from skimage import exposure

matched_img = exposure.match_histograms(img, reference_image)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Input Image')
axes[0].axis('off')
axes[1].imshow(reference_image, cmap='gray')
axes[1].set_title('Reference Image')
axes[1].axis('off')
axes[2].imshow(matched_img, cmap='gray')
axes[2].set_title('Histogram Matched Image')
axes[2].axis('off')
plt.show()
```